

Performance vs Efficiency in Social Media Hate Speech Detection: A Limited-Budget Comparison of Fine-Tuning Strategies

Zhi Li (u7640966)¹, Trinh Quang Minh Ngo (u7842952)¹, Aaron Nguyen (u7333216)¹, Hoang Bach Pham (u7861711)¹

Abstract

The rise of hateful and offensive content on social media has motivated research into automatic detection systems. This project compares two non-pretrained baselines, TF-IDF with logistic regression and Bi-LSTM, with five DistilBERT-based adaptation strategies for three-class hate speech classification on HateXplain: Frozen DistilBERT, full fine-tuning, LoRA, LP-FT, and EH-FT.

The comparison uses a shared dataset, preprocessing policy, validation objective, and final evaluation protocol. Hyperparameter optimisation uses validation macro-F1, and final test performance is reported over seeds 42, 43, and 44 using accuracy, macro-F1, macro-precision, and macro-recall. Efficiency is evaluated using training time and recorded peak GPU memory where available.

Rather than pursuing state-of-the-art accuracy, this study focuses on performance-cost trade-offs under a limited-budget setting. The results suggest that more complex adaptation strategies do not always provide practical gains. EH-FT achieved the highest mean accuracy and macro-F1, but with the highest recorded cost among the fine-tuning methods, while LoRA offered the clearest parameter- and memory-efficient trade-off among adaptive DistilBERT methods.

1 Introduction

Amid the growing use of social media, hate speech has become an important problem in online communication. Since hateful content can appear in short posts, replies, or targeted comments toward particular groups, automatic hate speech detection has become a relevant task in natural language processing. In this project, the task is treated as a three-class sequence classification problem on the HateXplain dataset, where each post is classified as *hatespeech*, *offensive*, or *normal* [13].

Hate speech is considered as devaluing minority groups and can increase an individual’s outgroup prejudice through frequent and repetitive exposure [13]. In contrast, offensive language may be commonly used within specific cultural communities, or specific media, but does not devalue minority groups; and such distinction is necessary as some research has argued that offensive language should

not be removed on social media, as opposed to hate speech [13].

With the development of machine learning and pre-trained language models, hate speech detection has moved beyond purely manual moderation or simple keyword filtering. Previous studies have compared traditional machine learning models, recurrent neural networks, and transformer-based models for hate speech detection [9, 6, 1]. However, within a limited-budget experimental setting, it is still useful to examine how different modelling and fine-tuning strategies behave when both classification performance and computational cost are considered.

For this purpose, our work compares two baseline models, TF-IDF with logistic regression (TF-IDF + LR), and Bi-LSTM, with five DistilBERT-based adaptation strategies: full fine-tuning (Full FT), linear probing (we also refer to as *Frozen DistilBERT*), LoRA, linear probing fine-tuning (LP-FT), and efficient head fine-tuning (EH-FT). DistilBERT is used as the shared pretrained backbone because it is smaller and faster than BERT-base while retaining comparable downstream performance [16]. The comparison is conducted under a shared data and evaluation protocol. Hyperparameter optimisation uses validation macro-F1 for model selection, while final test performance is reported over seeds 42, 43, and 44. Computational cost is measured using final training time and recorded peak GPU memory.

Therefore, under the given time and computational constraints, this project aims to assess whether more complex fine-tuning strategies lead to practically useful gains, and whether efficient or two-stage methods provide a better performance-cost trade-off. The main research question is:

How do different fine-tuning strategies compare in terms of classification performance and computational cost for hate speech detection on social media text?

To provide a more detailed analysis, this research question is divided into three sub-questions:

- How large is the performance gap between non-pretrained approaches, including TF-IDF + LR and Bi-LSTM, and pretrained DistilBERT-based approaches? Within the pretrained group, do Full FT, Frozen DistilBERT, LP-FT, LoRA, and EH-FT lead to meaningful differences in classification performance?
- When accounting for training time and recorded peak GPU memory, do alternative adaptation strategies (Frozen DistilBERT, LP-FT, LoRA, and EH-FT) offer a useful trade-off against Full FT?
- Do two-stage methods (LP-FT and EH-FT) outperform their single-stage counterparts (Full FT and LoRA) by a meaningful margin, and does the additional training complexity translate into practically useful gains?

¹Equal contribution; authors listed alphabetically by last name

2 Related Work

With the flourishing power of machine learning in classifying text over the past years, research has increasingly concentrated on applying different models on a wide variety of text datasets or using distinct fine-tuning strategies to improve the accuracy on downstream tasks within computational resource constraints.

2.1 Deep Learning Methods in Hate Speech Classification

Kapil et al. [9] compared hate speech detection for social media text using 13 different deep learning models, including CNN, LSTM, BiLSTM, and CharacterCNN. Conducting analysis on 3 different datasets, the research observed that LSTM and BiLSTM models managed to performed the best on the given datasets, with CharCNN adding to the boost of the performance.

Additionally, D’Sa et al. [6] suggested approaches on automatically identifying hate speech on social media with deep learning methods. They found fine-tuning of the pretrained BERT model results in better performance than the other feature-based approaches, with a 63% and 42% reduction rate in classification for binary-class and multi-class problems respectively, compared to the feature-based approaches.

With deep learning being a broad area of research, these are just two among numerous projects applying deep learning techniques in hate speech detection, which has further demonstrated this being a popular and active field of research.

2.2 Performance Comparison across Paradigms

Regarding comparison between models and fine-tuning strategies, Pan et al. [14] compared fine-tuning and context learning on different types of datasets including sexism and hate speech. In the experiment, LoRA fine-tuning was used with large language encoder-decoder models namely Mistral, Zephyr, StableBeluga, and Tulu-2. Specifically for hate speech detection, the Zephyr model with fine-tuning performed exceptionally with a recall value at 95.4%, despite recording a worse evaluation in the non-hate category. This work demonstrates how comparing the performance of different paradigms has been an active research area, particularly in the context of hate speech.

Additionally, Abusager et al. [1] analysed and evaluated 38 different models in hate speech detection. The comparison varied from traditional machine learning to modern transformer models with different performance findings. This motivates further research on how the model training method affects the results.

2.3 Efficiency-Performance Trade-Off

Gao et al. [7] examined the trade-off between effec-

tiveness and efficiency by comparing full fine-tuning with parameter-efficient strategies, including LoRA and QLoRA. Although full fine-tuning required more than 30% additional training time and 53% additional memory, the evaluation gap against the more efficient methods was small. This questions the assumption that a larger adaptation budget necessarily improves performance.

Raising more questions regarding the trade-off, Malik et al. [12] surveyed and analysed different strategies for hate speech detection. Notable examples include traditional classifiers (SVM, XGB, etc), deep models (CNN, MLP, etc) with GloVe embeddings, or deep models (BERT, Al-BERT, etc) with transformers-based pre-training. The result they found was that ‘the combination of BERT, ELECTRA, and Al-BERT and neural network-based classifiers perform consistently better than the other methods’ while also having the most computational complexity.

2.4 Two-Stage vs Single-Stage Adaption

Comparing two-stage methods and single-stage fine-tuning strategies remains an active research area. Tomihari and Sato [18] provide a theoretical analysis of two-stage adaptation. Their analysis supports the role of a linear-probing stage in preserving pretrained features before adapting a complex pretrained model such as DistilBERT.

Although their experiments use image data rather than hate-speech text, Kumar et al. [10] provide a relevant comparison between linear probing, full fine-tuning, and a combined strategy. They show that an initial linear-probing phase can help preserve pretrained features and improve out-of-distribution accuracy.

However, multi-stage adaptation does not always outperform single-stage tuning. Pezzuti et al. [15] report a setting in which two-stage strategies do not show statistically significant advantages, suggesting that a single fine-tuning phase can be sufficient in some scenarios.

2.5 Research Gap

All of the research above, together with the fast-moving field of deep learning, has laid a concrete foundation for our work in comparing deep learning models and fine-tuning methods within the context of hate speech detection on social media.

Nevertheless, with our distinct choices of the baseline models, and DistilBERT as the pretrained model to be applied with various fine-tuning strategies, our work aims to compare fine-tuning a pretrained model as DistilBERT, and cheaper baseline models including TF-IDF + LR and Bi-LSTM. Also, different aspects of comparison including the performance-efficiency trade-off with the fine-tuning strategies, and assessment on whether the two-stage methods outperform the single-stage methods, sets our research apart, especially considering that the work is applied to one particular field: hate speech detection on social media texts.

3 Methodology

This paper treats hate speech detection as a three-class sequence classification task in natural language processing. For each model, the input is a social media post, and the output is a label: *hatespeech*, *offensive*, or *normal*. We compare two non-pretrained baseline models with five DistilBERT-based adaptation strategies under a shared experimental setup. Classification performance is evaluated for all models, while training time is reported for all methods and recorded peak GPU memory is reported where it is available for GPU-backed methods.

3.1 Dataset

To train the models, we use the HateXplain dataset [13]. HateXplain contains tokenised social media posts from Twitter and Gab, excluding reposts and external media such as links, pictures, or videos. Each post is annotated by three qualified Amazon Mechanical Turk workers, and the final label is assigned by majority vote. Posts with complete annotator disagreement are excluded from the dataset. The dataset is already split, using stratified sampling to maintain class balance, into training, validation, and test sets, with 15383, 1922, and 1924 entries respectively, as shown in Table 1.

Class	Dataset Split			Total
	Training	Validation	Test	
Hateful	4748	593	594	5935
Offensive	4384	548	548	5480
Normal	6251	781	782	7814
Total	15383	1922	1924	19229

Table 1: HateXplain dataset splits and classes [13]

3.2 Pretrained Model

For the models that require fine-tuning, we use the pre-trained DistilBERT base model [16], a transformer-based distilled version of the Bidirectional Encoder Representations from Transformers (BERT) model. The baseline models are trained from scratch, while the fine-tuning methods adapt a pretrained model.

We use a smaller pretrained model because computational and memory requirements are central constraints in this project. DistilBERT is 40% smaller than BERT-base, has 60% faster inference time, and retains 97% of BERT’s performance on the reported language-understanding benchmark [16]. This makes it more suitable than BERT-base for comparing multiple fine-tuning strategies under a limited budget, while still providing a strong pretrained language representation.

The DistilBERT tokeniser converts text into sub-word token IDs. The model maps these IDs into 768-dimensional token and position embeddings, then pro-

cesses them through six Transformer encoder layers. Compared with BERT-base, DistilBERT removes token-type embeddings and the pooler, and reduces the number of Transformer layers from twelve to six while retaining the same hidden size [16, 5]. In this paper, the DistilBERT encoder is treated as the *backbone*. The task-specific *classification head* contains a linear pre-classifier layer ($768 \rightarrow 768$) and a linear classifier layer ($768 \rightarrow 3$), where 3 is the number of classes in HateXplain. The adaptation strategies differ mainly in which backbone, adapter, and classification head parameters are frozen or updated.

3.3 Experimented Models

Our non-pretrained baseline models are as follows.

TF-IDF + logistic regression (TF-IDF + LR). TF-IDF + LR is included as the sparse lexical baseline. TF-IDF represents each post with weighted word n-gram features, and logistic regression applies a simple linear classifier over those features. Feature-based logistic-regression classifiers have been used in hate and offensive speech detection [4]. In this research, TF-IDF + LR anchors the comparison against a computationally cheap, non-neural, and non-pretrained approach.

Bi-LSTM. Bi-LSTM is included as the non-pretrained neural baseline. Unlike TF-IDF + LR, it learns word embeddings from the training data and processes token sequences with a bidirectional recurrent encoder before the final linear classifier. It is therefore a useful middle point between sparse lexical classification and DistilBERT adaptation strategies: it models word order and left/right context, but it does not use pretrained Transformer representations. Recurrent models, including LSTM and Bi-LSTM, have been used in hate speech detection studies [9].

The DistilBERT adaptation methods are visualised in Figure 1, and are detailed as follows.

Full Fine-Tuning (Full FT). Full fine-tuning refers to updating all parameters of the DistilBERT model, including in both the backbone and classification head, the latter of which is randomly initialised [19]. Devlin et al. [5] establish that the BERT model, which DistilBERT is based on, can be fine-tuned on downstream tasks by updating all pretrained parameters. Full fine-tuning may be prone to catastrophic forgetting due to the back-propagation of chaotic gradients from the randomly initialised head [19].

Linear Probing (LP). With LP, only the classification head is trained, while the backbone is frozen entirely [19]. The risk of catastrophic forgetting for full fine-tuning motivates this method, where LP is a more stable method to find a classification head [19]. We also refer to this method as *Frozen DistilBERT* throughout this research.

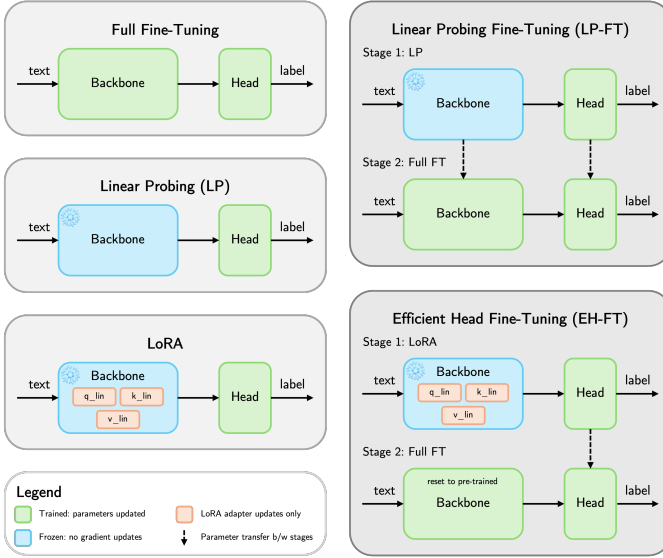


Figure 1: Visualisation of the DistilBERT adaptation methods with respect to freezing or updating the backbone, LoRA adapters, and classification head. The two-stage approaches are also visualised with the two individual stages.

Linear-Probing Fine-Tuning (LP-FT). LP-FT is a two-stage fine-tuning method where in stage 1, the backbone is frozen and only the head is trained, exactly as in LP, and in Stage 2, all parameters are unfrozen and are trained, exactly as in full fine-tuning [19].

Low-Rank Adaptation (LoRA). LoRA keeps the pretrained DistilBERT backbone largely fixed and trains low-rank adapter parameters in selected attention projection modules, together with the classification head [8]. In this implementation, LoRA therefore reduces the number of trainable parameters compared with Full FT while still allowing task-specific adaptation inside the attention layers.

Efficient Head Fine-Tuning (EH-FT). EH-FT is another two-stage method where in stage 1, the randomly initialised classification head and the backbone is trained using LoRA, and in stage 2, a fresh DistilBERT backbone along with the transferred head from stage 1 is fully fine-tuned [19]. Yang et al. [19] found that EH-FT provides a good initialisation of the head, which provides more stable gradient updates that guides the model to a local minimum close to the pretrained point, minimising the risk of catastrophic forgetting and overfitting.

3.4 Text Pre-processing

Before training the models, we first pre-process the data. To resolve the list of annotated labels, for each entry, we perform a majority vote. The provided HateXplain tokens are joined into a single text string, while rationales, tar-

gets, post IDs, and annotator metadata are excluded from model inputs. DistilBERT-based methods then use the DistilBERT tokeniser and padding policy so that special tokens such as [CLS] and [PAD] are handled consistently. TF-IDF + LR uses the same text field with a TF-IDF vectoriser followed by logistic regression, while Bi-LSTM builds a train-split word vocabulary before padding or truncating sequences. By default, DistilBERT supports a maximum sequence length of 512 [16]; however, HateXplain is a short social media text dataset [17], and this project fixes the maximum length at 128 to control computation. Automatic truncation is used only when a sequence exceeds this limit.

3.5 Hyperparameter Optimisation

HPO was treated as model selection, using validation macro-F1 (`eval_f1_macro`) as the objective and reserving the test split for the final evaluation only. This avoids repeatedly consulting the test set during model selection [3]. Because the budget was limited, the implementation used finite random-order search rather than a full grid; this is appropriate when only a subset of hyperparameters is expected to dominate performance [2].

The search spaces were compact and method-aware rather than identical across methods. They were chosen from hyperparameters expected to have high impact for each method, informed by the HateXplain task shape, common fine-tuning practice, method-specific literature, and the available compute budget; this follows the practical view that HPO for pretrained language-model fine-tuning depends strongly on the suitability of the search space and time budget [11]. Each space targeted vectoriser and regularisation choices for TF-IDF + LR; capacity, dropout, and learning rate for Bi-LSTM; learning rate for Frozen DistilBERT and Full FT; LoRA rank, target modules, and learning rate for LoRA; and stage-specific learning rates or LoRA rank for the two-stage methods. Protocol-level settings, such as the HateXplain split, no class weighting, maximum length 128, AdamW, linear scheduling, weight decay 0.01, warmup ratio 0.06, gradient clipping, and early stopping, were fixed to keep the comparison focused on method-specific choices. The resulting spaces should be interpreted as practical, evidence-informed search spaces rather than exhaustive or provably optimal ranges. The top two single-seed HPO configurations for each method were then rerun on validation seeds 42, 43, and 44; the higher mean validation macro-F1 selected the final setting.

3.6 Evaluation Metrics

For evaluating classification performance, we use accuracy, macro-F1, macro-precision, and macro-recall. Macro-F1 is the HPO selection metric because it gives equal weight to all three classes, while accuracy provides an appropriate, overall correctness measure given that the dataset classes

are balanced. Precision is more sensitive to false positives, whereas recall is more sensitive to false negatives.

For computational cost, we report training time for all methods and recorded peak GPU memory where this field is available. The memory value is taken from the aggregation code, which prioritises CUDA reserved memory. Therefore, this value is best interpreted as the GPU footprint observed in our implementation, rather than as a hardware-independent memory requirement.

4 Experiments

4.1 Hyperparameter Optimisation and Final Setting Choice

All tuning, confirmation, and final runs for the seven reported methods completed successfully. Table 2 reports the selected aggregate setting for each method, linked across HPO, confirmation, and final runs by configuration hash. Confirmation affected model selection: for Bi-LSTM, Frozen DistilBERT, LP-FT, and EH-FT, the final setting ranked second in single-seed HPO but first by three-seed confirmation mean. This supports using multiple validation seeds when neural-model rankings can vary by seed.

Method	Trials	Selected Tuned Setting
TF-IDF + LR	+ 24	$C = 1.0$, n-gram (1,2), min_df= 2, max_df= 0.9, 20000 features, sublinear TF
Bi-LSTM	20	embedding = 100, hidden = 256, dropout = 0.5, learning rate = 10^{-3}
Frozen DistilBERT	4	frozen backbone, head learning rate = 3×10^{-4} , 8 epochs
Full FT	4	full fine-tuning, learning rate = 2×10^{-5} , 4 epochs
LoRA	18	$r = 16$, $\alpha = 16$, target modules <code>q_lin</code> , <code>k_lin</code> , <code>v_lin</code> , <code>out_lin</code> , learning rate = 10^{-4}
LP-FT	9	stage 1 head learning rate = 10^{-4} for 5 epochs; stage 2 learning rate = 2×10^{-5} for 3 epochs
EH-FT	10	stage 1 LoRA $r = 4$, $\alpha = 4$, target modules <code>q_lin</code> , <code>v_lin</code> , learning rate = 2×10^{-4} for 5 epochs; stage 2 learning rate = 2×10^{-5} for 3 epochs

Table 2: Selected tuned settings for final seeds 42, 43, and 44. Trial counts exclude confirmation reruns; fixed protocol settings are described in the methodology.

The tuning-trial counts exclude confirmation reruns. They reflect a method-aware protocol: larger or more uncertain spaces, such as LoRA and Bi-LSTM, received more trials than smaller spaces such as Full FT and Frozen DistilBERT. The selected settings should therefore be read as tuned settings under a constrained protocol, not as globally optimal hyperparameters.

4.2 Performance Analysis

All approaches were run in Google Colab with an **NVIDIA A100-SXM4-40GB**, except for TF-IDF combined with Logistic Regression, which has been strictly run

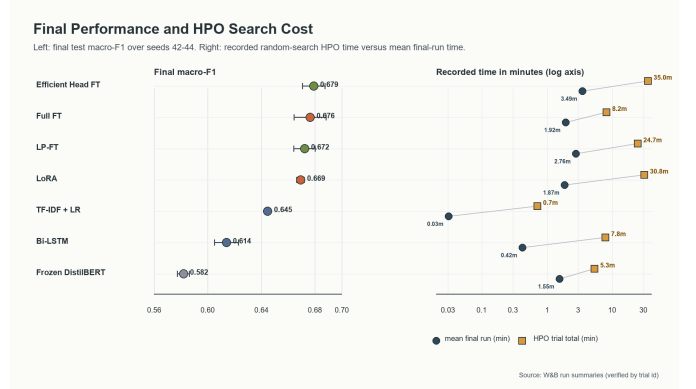


Figure 2: Final macro-F1 and recorded HPO time. Time labels are raw minutes on a log-scaled axis; HPO time excludes confirmation reruns and trial counts vary by method.

on **CPU**. After HPO, each selected setting was evaluated with seeds 42, 43, and 44; results are reported as mean $\pm$ standard deviation. The analysis is organised around the three research questions: the gap between non-pretrained and pretrained methods, the performance-cost trade-off among adaptation strategies, and whether two-stage adaptation clearly improves over single-stage fine-tuning.

Group	Model	Accuracy	Macro-F1	Macro-Prec	Macro-Recall
Baseline	TF-IDF + LR	0.668 ± 0.000	0.645 ± 0.000	0.661 ± 0.000	0.646 ± 0.000
	Bi-LSTM	0.628 ± 0.005	0.614 ± 0.009	0.617 ± 0.007	0.614 ± 0.008
Fine-Tuning	Frozen DistilBERT	0.598 ± 0.013	0.582 ± 0.005	0.587 ± 0.005	0.584 ± 0.008
	Full FT	0.687 ± 0.014	0.676 ± 0.012	0.677 ± 0.014	0.678 ± 0.009
	LP-FT	0.686 ± 0.006	0.672 ± 0.008	0.671 ± 0.008	0.676 ± 0.007
	LoRA	0.683 ± 0.001	0.669 ± 0.003	0.670 ± 0.003	0.671 ± 0.001
	EH-FT	0.693 ± 0.007	0.679 ± 0.008	0.680 ± 0.011	0.680 ± 0.006

Table 3: Final test performance over seeds 42, 43, and 44

Table 3 reports both accuracy and macro-F1, so performance is interpreted through overall correctness and class-balanced performance rather than accuracy alone. Figures 2 and 3 separate HPO search cost from selected-setting final-run cost. The four strongest DistilBERT adaptation methods are close in macro-F1, while their search and final-run costs differ more visibly.

Figure 2 is therefore evidence for transparency about search effort. A cost-equal HPO schedule was not used because the methods have different high-impact hyperparameters and different space sizes: forcing identical wall-clock or trial budgets could under-search higher-dimensional methods such as LoRA and the two-stage methods, or oversample smaller spaces such as Frozen DistilBERT and Full FT. The selected design instead

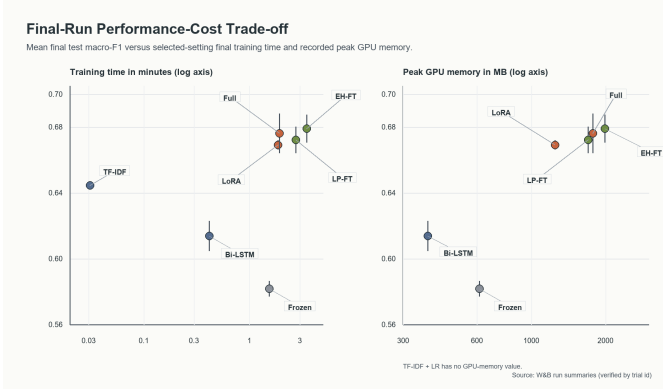


Figure 3: Final-run performance–cost trade-off for selected settings. Cost axes are log-scaled, and leader lines connect abbreviated method labels to points.

searches each method over a compact, literature- and implementation-informed space, then reports HPO time as a limitation when interpreting final performance.

4.2.1 Between Non-Pretrained and Pretrained Approaches

Within the *baseline* group, TF-IDF + LR was stronger, reaching 0.668 accuracy and 0.645 macro-F1. Bi-LSTM was lower but still above Frozen DistilBERT, with 0.628 accuracy and 0.614 macro-F1. Thus, the sparse lexical baseline remained a strong low-cost comparator, while the recurrent model did not close the gap to the best pre-trained adaptation methods.

Within the *fine-tuning* group, Frozen DistilBERT was the weakest approach among all seven methods (0.598 accuracy, 0.582 macro-F1). Simply using a pretrained encoder was therefore insufficient in this setting; the four DistilBERT methods that updated either the full model or adapter-related parameters all outperformed both baselines on accuracy and macro-F1.

4.2.2 Among Different Fine-Tuning Strategies

Among the five fine-tuning approaches, EH-FT had the highest mean accuracy and macro-F1 (0.693 and 0.679). But it did not clearly separated from Full FT (0.687/0.676), LP-FT (0.686/0.672), or LoRA (0.683/0.669): the largest gap inside this high-performing group was only about 0.010 in both metrics. These results are descriptive trends under the three-seed protocol, not decisive rankings. The clearest within-group difference was instead between Frozen DistilBERT and the four adaptive strategies.

4.3 Comparison Between Full FT and Alternative Adaptation Strategies

Table 4 shows that the alternative adaptation methods did not follow a single performance–cost pattern. Frozen Dis-

Group	Model	Accuracy	Macro-F1	Trainable Params	Time (Mins)	Memory (MB)
Full FT	Full FT	0.687 ± 0.014	0.676 ± 0.012	67.0M	1.923 ± 0.320	1773.333 ± 64.073
	Frozen	0.598 ± 0.013	0.582 ± 0.005	0.59M	1.549 ± 0.382	614.000 ± 38.105
Alternative	LP-FT	0.686 ± 0.006	0.672 ± 0.008	67.0M	2.756 ± 0.042	1701.333 ± 3.055
	LoRA	0.683 ± 0.001	0.669 ± 0.003	1.18M	1.869 ± 0.008	1246.667 ± 124.809
	EH-FT	0.693 ± 0.007	0.679 ± 0.008	67.0M	3.488 ± 0.021	1987.333 ± 54.123
	Full					

Table 4: Efficiency and performance comparison of fine-tuning methods. Accuracy, Macro-F1, training time, and peak reserved memory are reported as mean ± standard deviation across seeds.

tilBERT was the cheapest DistilBERT option in final runs, with 0.59M trainable parameters and 614 MB recorded peak memory, but its accuracy and macro-F1 were much lower than the other adaptation strategies. It is therefore efficient only in resource use, not in predictive performance.

LoRA provided the clearest parameter-efficient final-run trade-off. Compared with Full FT, it achieved similar accuracy (0.683 versus 0.687) and macro-F1 (0.669 versus 0.676), while training only 1.18M parameters instead of 67.0M and using lower recorded peak memory (1247 MB versus 1773 MB). Its mean runtime was also close to Full FT. By contrast, LP-FT and EH-FT did not reduce final-run time in this implementation. EH-FT achieved the highest mean scores, but it also recorded the largest runtime and memory among the fine-tuning methods, so its advantage is better interpreted as a small performance gain with additional computational cost.

4.4 Comparison Between Single-Stage and Two-Stage Methods

Group	Model	Accuracy	Macro-F1	Trainable Params	Time (Mins)	Memory (MB)
Single-stage	Full FT	0.687 ± 0.014	0.676 ± 0.012	67.0M	1.923 ± 0.320	1773.333 ± 64.073
	LoRA	0.683 ± 0.001	0.669 ± 0.003	1.18M	1.869 ± 0.008	1246.667 ± 124.809
Two-stage	LP-FT	0.686 ± 0.006	0.672 ± 0.008	67.0M	2.756 ± 0.042	1701.333 ± 3.055
	EH-FT	0.693 ± 0.007	0.679 ± 0.008	67.0M	3.488 ± 0.021	1987.333 ± 54.123

Table 5: Comparison of single-stage and two-stage fine-tuning methods. Accuracy, Macro-F1, training time, and peak reserved memory are reported as mean ± standard deviation across seeds.

Table 5 shows that the two-stage methods did not clearly dominate the single-stage methods. Across the four high-performing strategies, accuracy ranged from 0.683 to

0.693 and macro-F1 from 0.669 to 0.679, which is small relative to the three-seed variability reported in Table 3. The larger difference was computational cost. EH-FT had the highest mean final-run time at 3.488 minutes, followed by LP-FT at 2.756 minutes, whereas Full FT and LoRA were faster at 1.923 and 1.869 minutes.

The recorded memory pattern gives a similar conclusion. EH-FT had the highest recorded peak memory at 1987 MB. LP-FT was close to Full FT (1701 MB versus 1773 MB), while LoRA used the lowest recorded memory among these four methods at 1247 MB. Thus, the two-stage procedures did not provide a general efficiency advantage in this particular context of hate speech texts; their value depends on whether the small observed performance differences are worth the additional training cost.

5 Conclusion and Discussion

5.1 Conclusion

Within the constrained time and compute budget, this project compared two non-pretrained baselines with five DistilBERT adaptation strategies on HateXplain. For the first research question, TF-IDF + LR remained a strong lightweight baseline and can still be considered a reasonable low-cost choice for this task. Bi-LSTM also exceeded Frozen DistilBERT; however, the four adaptive DistilBERT methods achieved the best accuracy and macro-F1. Within this adaptive group, EH-FT recorded the highest mean scores, but its advantage over Full FT, LP-FT, and LoRA was small under the three-seed protocol. For the cost-focused comparison, Full FT remained a strong reference point, while LoRA offered the clearest parameter- and memory-efficient final-run trade-off. Frozen DistilBERT was cheaper but lost too much performance, and the two-stage methods required more runtime without a clear efficiency advantage over single-stage fine-tuning. Overall, the results do not show that two-stage adaptation outperforms single-stage fine-tuning by a meaningful practical margin in this setting. These findings should be read as evidence from a limited time, method-aware, limited-budget HPO protocol rather than as a statistically decisive or equal-cost ranking of fine-tuning strategies.

5.2 Limitations and Future Work

This evaluation remains bounded by its limited-budget, method-aware design. The final comparison uses only seeds 42, 43, and 44, so small differences among the strongest methods should be interpreted cautiously. The HPO budget was intentionally method-aware rather than equalised by GPU hours, since identical budgets could under-search larger spaces and over-search smaller ones; therefore, the rankings reflect both method behaviour and the chosen search protocol. Runtime and recorded peak GPU memory are also Colab implementation-level measurements rather than hardware-independent require-

ments. Additional diagnostic outputs, including per-class metrics, prediction files, and material for confusion-matrix or error analysis, were produced, but only the per-class F1 summary is included here due to the page limit.



Figure 4: Mean test F1 by class over final seeds 42, 43, and 44.

Figure 4 suggests that the remaining error is partly class-specific: the offensive class has the lowest mean F1 across all methods, while hate and normal are consistently higher. Future work should report the full confusion-matrix and error-analysis results, examine which class boundaries involving offensive content drive these errors, and test class-aware objectives or targeted data augmentation. A separate equal-cost or budget-sensitivity HPO study would also be needed to test whether the small differences among Full FT, LoRA, LP-FT, and EH-FT remain under alternative search-cost assumptions.

Declaration and Acknowledgement

We declare the use of the following Generative AI tools: ChatGPT, Gemini, and Claude. These tools were *not* used to write, edit, or optimise the report. The report content, analysis, interpretation, and final writing are our own work. Furthermore, the ideas, experimental design, model logic, and base model implementations are our original work.

Generative AI tools were used only to support the coding process in two ways. First, they were used to help debug implementation issues. Second, they were used to assist with integrating the models that we had written ourselves into a shared experimental pipeline and repository structure and also help to document the experiment process. This integration process was not fully generated by AI; we were actively involved throughout the development process, including developing, reviewing, modifying, and understanding the code. We verified the correctness of the integrated code by reading and understanding it thoroughly, checking that the experiments were run properly, and confirming that the integrated pipeline produced behaviour and trends consistent with our expectations and with findings reported in relevant papers.

References

- [1] Mahmoud Abusaqer, Jamil Saquer, and Hazim Shatnawi. Efficient Hate Speech Detection: Evaluating 38 Models from Traditional Methods to Transformers. In *Proceedings of the 2025 ACM Southeast Conference, ACMSE 2025*, page 203–214, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400712777. doi: 10.1145/3696673.3723061. URL <https://doi.org/10.1145/3696673.3723061>.
- [2] James Bergstra and Yoshua Bengio. Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13(10):281–305, 2012. URL <https://www.jmlr.org/papers/v13/bergstra12a.html>.
- [3] Gavin C. Cawley and Nicola L. C. Talbot. On Overfitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL <https://www.jmlr.org/papers/v11/cawley10a.html>.
- [4] Thomas Davidson, Dana Warmusley, Michael Macy, and Ingmar Weber. Automated Hate Speech Detection and the Problem of Offensive Language. In *Proceedings of the Eleventh International AAAI Conference on Web and Social Media*, pages 512–515, 2017. doi: 10.1609/icwsm.v11i1.14955.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR*, abs/1810.04805, 2018. URL <http://arxiv.org/abs/1810.04805>.
- [6] Ashwin Geet D’Sa, Irina Illina, and Dominique Fohr. BERT and fastText Embeddings for Automatic Detection of Toxic Speech. *2020 International Multi-Conference on: “Organization of Knowledge and Advanced Technologies” (OCTA)*, pages 1–5, 2020. doi: 10.1109/OCTA49274.2020.9151853.
- [7] Zhengguang Gao, Shenjia Jing, and Lihong Zhang. A Study on the Application of Large Language Models Based on LoRA Fine-Tuning and Difficult-Sample Adaptation for Online Violence Recognition. *Symmetry*, 17:1310, 08 2025. doi: 10.3390/sym17081310.
- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models, 2021. URL <https://arxiv.org/abs/2106.09685>.
- [9] Prashant Kapil, Asif Ekbal, and Dipankar Das. Investigating Deep Learning Approaches for Hate Speech Detection in Social Media. *arXiv preprint arXiv:2005.14690*, 2020. URL <https://arxiv.org/abs/2005.14690>.
- [10] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution, 2022. URL <https://arxiv.org/abs/2202.10054>.
- [11] Xueqing Liu and Chi Wang. An Empirical Study on Hyperparameter Optimization for Fine-Tuning Pre-trained Language Models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 2286–2300, 2021. doi: 10.18653/v1/2021.acl-long.178. URL <https://aclanthology.org/2021.acl-long.178/>.
- [12] Jitendra Singh Malik, Hezhe Qiao, Guansong Pang, and Anton van den Hengel. Deep learning for hate speech detection: a comparative study. *International Journal of Data Science and Analytics*, 20:3053–3068, 2025. doi: 10.1007/s41060-024-00650-6.
- [13] Binny Mathew, Punyajoy Saha, Seid Muhie Yimam, Chris Biemann, Pawan Goyal, and Animesh Mukherjee. HateXplain: A Benchmark Dataset for Explainable Hate Speech Detection, 2022. URL <https://arxiv.org/abs/2012.10289>.
- [14] Ronghao Pan, José García-Díaz, and Rafael Valencia-García. Comparing Fine-Tuning, Zero and Few-Shot Strategies with Large Language Models in Hate Speech Detection in English. *Computer Modeling in Engineering & Sciences*, 140(3):2849–2868, 2024. doi: 10.32604/cmescs.2024.050474.
- [15] Francesca Pezzuti, Sean MacAvaney, and Nicola Tonellotto. Exploring the Effectiveness of Multi-stage Fine-tuning for Cross-encoder Rerankers. *ArXiv*, abs/2503.22672, 2025. doi: 10.48550/arxiv.2503.22672.
- [16] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *ArXiv*, abs/1910.01108, 2019.
- [17] Tanmay Sen, Ansuman Das, and Mrinmay Sen. HateTinyLLM : Hate Speech Detection Using Tiny Large Language Models, 2024. URL <https://arxiv.org/abs/2405.01577>.
- [18] Akiyoshi Tomihari and Issei Sato. Understanding Linear Probing then Fine-tuning Language Models from NTK Perspective, 2024. URL <https://arxiv.org/abs/2405.16747>.
- [19] Zhuoyi Yang, Ming Ding, Yanhui Guo, Qingsong Lv, and Jie Tang. Parameter-Efficient Tuning Makes a Good Classification Head, 2023. URL <https://arxiv.org/abs/2210.16771>.